Final Exam — Sample B

Name: _____

Software Engineering — UATX — Spring 2026

Format	Pencil and paper. No computer, phone, notes, or coding agent.
Time	150 minutes.
Length	Seven questions, one to two per category.
Coverage	Lectures 5.1 through 10.1 (JS/TS and the browser, React, deployment, Docker, CI/CD, concurrency, security, agents), plus assignments 3 and 4.
Categories	Trace-through, spot the bug, specification, code review, short design.
Grading	Curved. The exam is intentionally hard; expect to leave some parts unfinished.

HOW THIS EXAM THINKS ABOUT AGENTS

The whole semester has leaned into coding agents as the way real engineers work, so an exam that asks you to write working code from scratch on paper would punish you for taking the course seriously. Instead, every question is built around a skill the agent can't do for you: reading code carefully, finding bugs by inspection, writing the spec you'd hand to the agent, judging which of two implementations is better, and making design calls under uncertainty.

HOW TO USE THIS SAMPLE

This is a representative exam, not the actual one — the real final will have seven questions in the same five categories at roughly the same difficulty. Solutions are in a separate document; try the questions cold first, then check.

Question 1

Trace-through

This component renders once on mount, then the user clicks the button exactly once. Give the full console output in order, and answer the questions below.

```
function Likes() {
  const [likes, setLikes] = useState(3);

  useEffect(() => {
    console.log("effect:", likes);
  }, [likes]);

  function onClick() {
    setLikes(likes + 1);
    setLikes(likes + 1);
    console.log("handler:", likes);
  }

  return <button onClick={onClick}>{likes}</button>;
}
```

1. List everything the console logs, in order, from the initial mount through the one click.
2. After that single click, what number does the button show — 4 or 5? Explain why.
3. The dev wants one click to add 2 (so the button goes to 5). Give the one-line change and say why it works.

Question 2

Trace-through

The script runs to completion. Give the value printed at A, B, C, and D, then answer the follow-up.

```
const nums = [3, 1, 2];
const sorted = nums.sort((a, b) => b - a);
console.log("A:", nums, sorted);                                  // ___

const prices = [
  { item: "pen", cents: 150 },
  { item: "pad", cents: 320 },
  { item: "pen", cents: 150 },
];
const total = prices.reduce((s, p) => s + p.cents, 0);
console.log("B:", total);                                         // ___

const names = ["ann", "bob", "cat"];
const upper = names.map(n => n.toUpperCase());
console.log("C:", names, upper);                                  // ___

console.log("D:", [1, 2, 3] + [4, 5], typeof NaN, 0.1 + 0.2 === 0.3); // ___
```

1. Give A, B, C, and D.
2. One of these lines quietly mutates a variable you might not expect. Which one, and how would you avoid the surprise?

Question 3

Spot the bug

Your agent produced this live-search box. It works when you type slowly on a good connection. Find at least three defects; for each, one sentence on what goes wrong and one on the fix.

```
const box = document.querySelector("#q") as HTMLInputElement;
const out = document.querySelector("#out") as HTMLUListElement;

box.addEventListener("input", async () => {
  const res = await fetch("/api/search?q=" + box.value);
  const hits = await res.json();
  out.innerHTML = "";
  for (const h of hits) {
    const li = document.createElement("li");
    li.innerHTML = h.name;
    out.append(li);
  }
});
```

Question 4

Spot the bug

Your agent produced this endpoint to read a direct message. It authenticates the caller and parameterizes its query. Read it carefully.

```
@app.get("/api/messages/{message_id}")
def get_message(message_id: int, user: User = Depends(current_user)):
    with engine.connect() as conn:
        row = conn.execute(
            text("SELECT * FROM messages WHERE id = :id"),
            {"id": message_id},
        ).mappings().first()
    if row is None:
        raise HTTPException(404)
    return dict(row)
```

1. The endpoint authenticates the caller. Why is that not enough here?
2. It has an access-control hole. Describe it, give a concrete request that exploits it, and give the fix.
3. `SELECT *` then `return dict(row)` creates a second, quieter problem. Name it and the fix.

Question 5

Specification

Feature request to your agent: “Show the feed of posts.” The first cut is below and it looks fine in a demo with a handful of posts. Before you let it ship, write the spec you’d hand back. Do not write code.

```
function Feed({ posts }: { posts: Post[] }) {
  return (
    <ul>
      {posts.map(p => (
        <li key={p.id}>
          <strong>{p.author}</strong>: {p.message}
          <span>{p.replies.length} replies</span>
        </li>
      ))}
    </ul>
  );
}
```

1. The rendering / display states this component must handle — name them.
2. Two things it gets wrong or omits for real data (look hard at empty results and partially-loaded posts).
3. One product decision that is ambiguous, and the call you would make.

Question 6

Code review

A teammate opens a PR adding this CI workflow. The repo is a FastAPI backend plus a React/TypeScript frontend (with Vitest and Playwright tests under `frontend/`), deployed on Railway. The team believes that once this merges, CI protects `main`.

```
# .github/workflows/ci.yml
name: CI
on:
  push:
    branches: [main]
jobs:
  backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/setup-python@v6
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt
      - run: pytest
      - run: echo "Deploying with ${{ secrets.RAILWAY_TOKEN }}" && railway up
```

1. The team thinks merging to `main` is gated by this check. Give two reasons it isn't, and the fix.
2. Given a React + Vitest + Playwright frontend, what does this workflow never test? Why is that "green check" misleading?
3. Two things are wrong with the last step. Name both and say what you'd do instead.

Question 7

Short design

Your agent set the BBS up for production on Railway with a Supabase Postgres. Relevant pieces of the setup are below. On `push` to `main`, Railway runs `pip install -r requirements.txt` and then `uvicorn app:app`. The frontend is built on the developer's laptop and the resulting `frontend/dist` is committed to the repo. There are no migrations; the developer edits the Supabase tables by hand in the dashboard.

```
# app.py (excerpt)
app = FastAPI()
app.mount("/", StaticFiles(directory="frontend/dist", html=True))

@app.get("/api/posts")
def list_posts():
    ...

DATABASE_URL = "postgresql://app:hunter2@db.supabase.co:5432/postgres"
engine = create_engine(DATABASE_URL)
```

1. After deploy, every call to `/api/posts` returns a 404 instead of your JSON — your handler never runs. Why, and what's the fix? (It's the same-origin static-serving trick, done wrong.)
2. There's a leaked secret here. Identify it, say how to inject it properly, and say what else you must do because it was already committed.
3. The build-and-schema process has two problems that cause "works on my machine" and dev/prod drift. Name both and give the right approach for each.